GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering & Technology

Subsequence Problem Breakdown

Case study: Huffman Coding and the Longest Common Subsequence (LCS) problem

Student Name	_____	Enrollment No.	_____
Subject	Advanced Data Structure (METCE11502)	Semester	1 (M.Tech)
Activity	ALA 3	Marks	10

1. Aim

To analyse, as a case study, the application of the **Huffman Coding algorithm** and the **Longest Common Subsequence (LCS) problem**; to investigate real-world scenarios in which these algorithms are used; to implement the solutions (LCS using **dynamic programming**) and to evaluate their effectiveness in various contexts. All implementations were written in Python and every number in this report comes from running that code.

2. Introduction

	Huffman Coding	Longest Common Subsequence
Problem	Given symbol frequencies, assign binary codes so that the encoded message is as short as possible.	Given two sequences X and Y, find the longest sequence that appears in both in the same order (not necessarily contiguously).
Technique	Greedy algorithm with a min-heap (priority queue).	Dynamic programming over a table of prefixes.
Key property	Optimal prefix-free code (no code is a prefix of another, so decoding is unambiguous).	Optimal substructure and overlapping sub-problems.
Complexity	O(k log k) for k distinct symbols	O(m·n) time and space for sequences of lengths m and n
Typical use	Lossless compression (ZIP/gzip/PNG, JPEG, MP3)	diff / version control, DNA comparison, plagiarism detection

Note: Huffman's algorithm is a **greedy** algorithm, not a dynamic-programming algorithm; the dynamic-programming part of this case study is the LCS problem. Both problems are solved by combining optimal solutions of smaller sub-problems.

3. Part A – Huffman Coding

3.1 Algorithm

- Count the frequency of every symbol.
- Create one leaf node per symbol and put all nodes into a min-heap keyed by frequency.
- Repeat until one node remains: remove the two nodes with the smallest frequencies, create a parent whose frequency is their sum, and insert it back.

- Read the code of each symbol from the root: left edge = 0, right edge = 1. Frequent symbols end up near the root (short codes).
- Decoding: follow the bits from the root; on reaching a leaf output its symbol and restart at the root.

Why it is optimal: (i) *Greedy choice* – the two least frequent symbols can always be made siblings at the deepest level of some optimal tree (exchange argument); (ii) *optimal substructure* – merging them into one symbol of combined frequency leaves a smaller problem whose optimal tree gives an optimal tree for the original. Since the code is prefix-free, no separators between codes are needed.

Huffman tree for ABRACADABRA (edge labels: 0 = left, 1 = right)

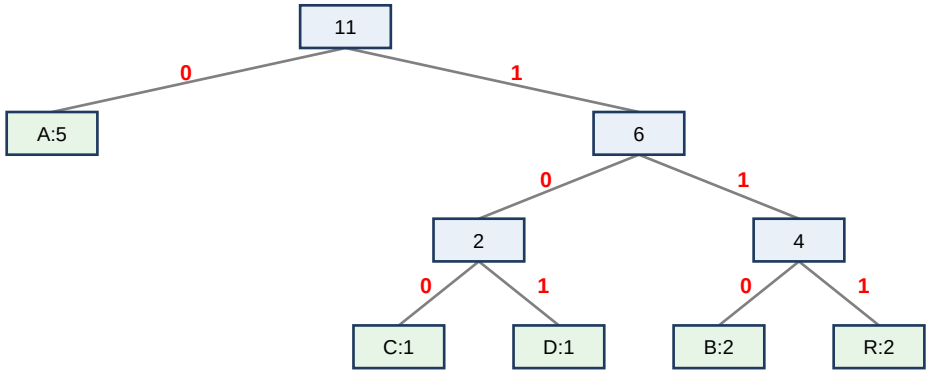


Figure 1: Huffman tree built by the program. Codes: A = 0, C = 100, D = 101, B = 110, R = 111.

3.2 Worked example (program output)

```
Text : ABRACADABRA
Frequencies: {'A': 5, 'B': 2, 'C': 1, 'D': 1, 'R': 2}
  A (freq 5) -> 0
  B (freq 2) -> 110
  C (freq 1) -> 100
  D (freq 1) -> 101
  R (freq 2) -> 111
Encoded bits : 01101110100010101101110
Size: 88 bits (8-bit ASCII)  ->  23 bits (Huffman)
Decoded      : ABRACADABRA | round trip OK: True
```

The 11-character word needs 88 bits in 8-bit ASCII but only 23 bits with Huffman codes; the most frequent letter (A, 5 times) receives a 1-bit code.

3.3 Real-world scenarios tested

Three data sets were encoded and decoded; in every case the decoded text was verified to equal the original. The baseline is the fixed-length encoding (8 bits per character for text, 2 bits per base for DNA). *Entropy* is the theoretical lower bound on the average number of bits per symbol.

Data set	Length	Distinct symbols	Fixed-length bits	Huffman bits	Avg bits / symbol	Entropy	Space saved
English paragraph	680	32	5440	2893	4.25	4.23	46.8%
Skewed DNA (A45% C30% G15% T10%)	2000	4	4000	3620	1.81	1.79	9.5%
Uniform DNA (25% each)	2000	4	4000	4000	2.00	2.00	0.0%

Most frequent symbols of the English paragraph and their codes:

Symbol	Frequency	Code	Length (bits)
' '	106	110	3

Symbol	Frequency	Code	Length (bits)
'e'	67	001	3
's'	53	1110	4
't'	51	1011	4
'o'	44	1001	4
'a'	42	0111	4
'n'	42	1000	4
'i'	38	0101	4

Rarest symbols (they receive the longest codes):

Symbol	Frequency	Code	Length (bits)
'A'	1	011000001	9
'x'	1	00010000	8
'C'	1	011000010	9
'H'	1	011000011	9

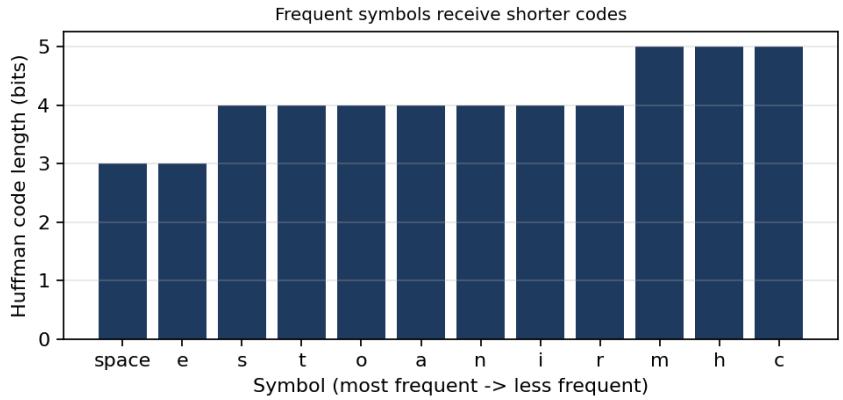


Figure 2: Code length grows as symbol frequency falls (space and 'e' get 3 bits, the rarest symbols 8–9 bits).

3.4 Evaluation of effectiveness

- **Skewed data compresses well.** The English paragraph shrank by 46.8% relative to 8-bit characters, and the average code length (4.25 bits) is within 0.02 bit of the entropy limit (4.23).
- **Mild skew gives mild gains.** DNA with frequencies 45/30/15/10% saved 9.5% (1.81 bits per base against an entropy of 1.79).
- **Uniform data cannot be compressed.** With four equally likely bases Huffman needs exactly 2 bits per base – 0.0% saving.
- **Overhead:** the decoder needs the code table (or frequencies). For very short messages this can outweigh the saving; the table is not counted in the figures above.
- **Limits:** Huffman uses a whole number of bits per symbol, so it can lose up to 1 bit per symbol versus entropy (arithmetic coding removes this limit), and a static code does not adapt to changing statistics (adaptive Huffman does).
- **Where it is used:** the entropy-coding stage of DEFLATE (ZIP, gzip, PNG), of JPEG, and of MP3, usually after a model or dictionary step (for example LZ77 in DEFLATE) has made the symbol distribution skewed.

4. Part B – Longest Common Subsequence (Dynamic Programming)

4.1 Problem and recurrence

A *subsequence* keeps the original order but may skip elements (e.g. GTAB is a subsequence of AGGTAB). The LCS of X (length m) and Y (length n) is found by filling a table L[i][j] = length of the LCS of the first i characters of X and the first j characters of Y:

```
L[i][0] = 0 and L[0][j] = 0                    (empty prefix)
L[i][j] = L[i-1][j-1] + 1              if X[i] == Y[j]   (characters match)
L[i][j] = max( L[i-1][j], L[i][j-1] )  otherwise         (skip a character of X or of Y)

Answer: L[m][n].  The string itself is recovered by walking back from L[m][n] to L[0][0].
```

Why dynamic programming? The plain recursion re-solves the same pairs of prefixes many times (overlapping sub-problems). Storing each answer once reduces the work from exponential to O(m·n).

4.2 Worked example (program output)

```
X = AGGTAB   Y = GXTXAYB
DP table L[i][j] (rows: prefixes of X, columns: prefixes of Y)
      -  G  X  T  X  A  Y  B
  -   0  0  0  0  0  0  0  0
  A   0  0  0  0  0  1  1  1
  G   0  1  1  1  1  1  1  1
  G   0  1  1  1  1  1  1  1
  T   0  1  1  2  2  2  2  2
  A   0  1  1  2  2  3  3  3
  B   0  1  1  2  2  3  3  4
LCS length = 4 | LCS = GTAB
```

The bottom-right cell is 4, and backtracking gives the common subsequence **GTAB**.

4.3 Real-world scenarios tested

Scenario 1: file comparison (diff / version control)

Treat each *line* as one symbol. Lines in the LCS are unchanged; lines of the old file not in the LCS are deletions ('–') and lines of the new file not in the LCS are insertions ('+'). This is the idea behind `diff` and `git diff`. Program output for two versions of a small function:

```
- def total(items):
+ def total(items, tax):
      t = 0
      for i in items:
          t = t + i.price
+     t = t * (1 + tax)
      return t
- print(total(cart))
+ print(total(cart, 0.18))
```

Scenario 2: DNA sequence similarity

Two DNA fragments of 32 bases each:

```
S1 = ATGGCGTACCTAGGCTTACGATCGGATCCGTA
S2 = ATGCCGTACTTAGGCATACGTTCGGACCGTTA
LCS = ATGCGTACTAGGCTACGTCGGACCGTA   (length 27)
```

Similarity = 2 × LCS / (|S1| + |S2|) = **84.4%**. The longer the common subsequence, the more closely related the sequences are (bioinformatics tools use closely related dynamic-programming alignment methods).

Scenario 3: text similarity / plagiarism check

Using *words* as symbols, the sentences “the quick brown fox jumps over the lazy dog near the river bank” (13 words) and “a quick brown fox leaps over the lazy dog close to the river bank” (14 words) share an LCS of 10 words, giving a similarity of **74.1%**. Rewording a few words does not hide the copying, because the order of the remaining words is preserved.

4.4 Implementation (Python)

Complete source code of both parts (`huff_lcs.py`): Huffman tree construction, encoding and decoding; naive recursive LCS, dynamic-programming LCS with backtracking, a two-row space-optimised version, and the LCS-based line diff.

```
import heapq
from collections import Counter


# ============================================================================
#  HUFFMAN CODING
# ============================================================================
class HNode:
    def __init__(self, freq, symbol=None, left=None, right=None):
        self.freq, self.symbol, self.left, self.right = freq, symbol, left, right


def build_huffman_tree(text):
    freq = Counter(text)
    heap = []
    for i, (sym, f) in enumerate(sorted(freq.items())):
        heapq.heappush(heap, (f, i, HNode(f, sym)))          # (freq, tie-breaker, node)
    counter = len(heap)
    if len(heap) == 1:                                        # only one distinct symbol
        f, _, n = heap[0]
        return HNode(f, None, n, None), freq
    while len(heap) > 1:
        f1, _, a = heapq.heappop(heap)                        # two least frequent
        f2, _, b = heapq.heappop(heap)
        heapq.heappush(heap, (f1 + f2, counter, HNode(f1 + f2, None, a, b)))
        counter += 1
    return heap[0][2], freq


def build_codes(root):
    codes = {}

    def walk(node, prefix):
        if node.symbol is not None:
            codes[node.symbol] = prefix or "0"
            return
        if node.left:
            walk(node.left, prefix + "0")
        if node.right:
            walk(node.right, prefix + "1")
    walk(root, "")
    return codes


def huffman_encode(text):
    root, freq = build_huffman_tree(text)
    codes = build_codes(root)
    bits = "".join(codes[ch] for ch in text)
    return bits, root, codes, freq


def huffman_decode(bits, root):
    out, node = [], root
    for b in bits:
        node = node.left if b == "0" else node.right
        if node.symbol is not None:
            out.append(node.symbol)
            node = root
        elif node.left is None and node.right is None:
            node = root
```

```
    return "".join(out)


# ======================================================================
#  LONGEST COMMON SUBSEQUENCE
# ======================================================================
def lcs_naive(a, b, counter):
    """Plain recursion: exponential time because sub-problems are recomputed."""
    counter[0] += 1
    if not a or not b:
        return 0
    if a[-1] == b[-1]:
        return 1 + lcs_naive(a[:-1], b[:-1], counter)
    return max(lcs_naive(a[:-1], b, counter), lcs_naive(a, b[:-1], counter))


def lcs_dp(a, b):
    """Bottom-up dynamic programming, O(m*n) time and space. Returns (table, lcs_string)."""
    m, n = len(a), len(b)
    L = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                L[i][j] = L[i - 1][j - 1] + 1                # characters match
            else:
                L[i][j] = max(L[i - 1][j], L[i][j - 1])      # skip one character
    i, j, out = m, n, []
    while i > 0 and j > 0:                                   # backtrack to recover the string
        if a[i - 1] == b[j - 1]:
            out.append(a[i - 1]); i -= 1; j -= 1
        elif L[i - 1][j] >= L[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return L, "".join(reversed(out))


def lcs_length_two_rows(a, b):
    """Space-optimised: only two rows are kept, O(min(m,n)) memory (length only)."""
    if len(b) > len(a):
        a, b = b, a
    prev = [0] * (len(b) + 1)
    for ch in a:
        cur = [0]
        for j, c in enumerate(b, 1):
            cur.append(prev[j - 1] + 1 if ch == c else max(prev[j], cur[j - 1]))
        prev = cur
    return prev[-1]


def diff_lines(old, new):
    """Line-based diff built on LCS (the idea behind 'diff' and 'git diff')."""
    L, _ = lcs_dp(old, new)
    i, j, out = len(old), len(new), []
    while i > 0 or j > 0:
        if i > 0 and j > 0 and old[i - 1] == new[j - 1]:
            out.append("  " + old[i - 1]); i -= 1; j -= 1
        elif j > 0 and (i == 0 or L[i][j - 1] >= L[i - 1][j]):
            out.append("+ " + new[j - 1]); j -= 1
        else:
            out.append("- " + old[i - 1]); i -= 1
    return list(reversed(out))
```

4.5 Evaluation: naive recursion versus dynamic programming

Random DNA strings of equal length n were used (average of 12 pairs for each n). The naive recursion counts function calls; dynamic programming fills $(n+1)^2$ cells.

n	Naive recursion: avg calls	DP: table cells	Naive time (ms)	DP time (ms)
4	58	25	0.01	0.007
6	255	49	0.05	0.013
8	2569	81	0.55	0.023

n	Naive recursion: avg calls	DP: table cells	Naive time (ms)	DP time (ms)
10	13672	121	2.80	0.028
11	53056	144	11.11	0.045
12	84125	169	17.45	0.057

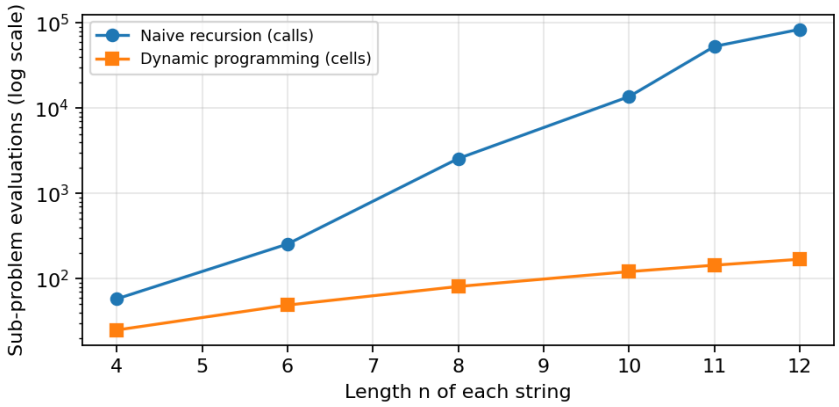


Figure 3: The number of sub-problems solved by the naive recursion explodes, whereas the DP table grows only quadratically.

Dynamic programming handles much longer inputs. The table below shows larger sequences; the second implementation keeps only two rows of the table and returns the length only.

n	Table cells	LCS length	DP with full table (s)	DP two rows (s)
500	251001	314	0.07	0.07
1000	1002001	647	0.28	0.18
2000	4004001	1293	1.03	0.77

- **Time:** O(m·n) – doubling n roughly quadruples the time (0.28 s → 1.03 s in this implementation for n = 1000 → 2000).
- **Space:** the full table needs O(m·n) memory (4 million cells for n = 2000), which is the practical limit for long inputs. Keeping two rows reduces memory to O(n) when only the length is needed; Hirschberg's divide-and-conquer method recovers the actual subsequence in linear space.
- **Scalability in practice:** professional diff tools do not fill the whole table; they use the Myers O(ND) algorithm, which is fast when the two files are similar (D, the number of differences, is small).
- **Limitation:** LCS finds *a* longest common subsequence, which need not be unique, and it measures order-preserving similarity only (not moved blocks of text).

5. Summary of the Case Study

Scenario	Algorithm	Measured outcome	Effective when / limits
Compress English text	Huffman (greedy)	46.8% smaller than 8-bit text; 4.25 bits/char vs entropy 4.23	Skewed symbol frequencies; code table overhead for tiny files
Compress skewed DNA	Huffman	9.5% smaller than 2 bits/base	Gain depends on skew; bounded by entropy
Compress uniform DNA	Huffman	0% saving	No gain when all symbols are equally likely
Compare two code versions	LCS (DP) on lines	3 insertions / 2 deletions found; 4 lines kept	Small and similar files; use Myers for big files
Compare DNA fragments	LCS (DP) on bases	LCS 27 of 32 bases (84.4% similarity)	Memory O(m·n) for long genomes
Detect copied text	LCS on words	74.1% similarity after rewording	Detects order-preserving copying only

6. Conclusion

Huffman coding and LCS are two classic algorithms that solve optimisation problems by building on smaller optimal solutions. Huffman's greedy algorithm produced optimal prefix codes that came within 0.02 bit of the entropy limit on text, but gave no gain on uniformly distributed data, showing that its effectiveness depends on the data. The dynamic-programming solution of LCS reduced the work from exponential (about 84,125 recursive calls on average for strings of only 12 characters) to a table of just 169 cells, and solved practical problems such as file comparison, DNA similarity and plagiarism detection. For very large inputs memory becomes the bottleneck, which is addressed by two-row, Hirschberg and Myers-type algorithms.

References

- D. A. Huffman, “A method for the construction of minimum-redundancy codes”, *Proceedings of the IRE*, 40(9), 1952.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press (Greedy algorithms; Longest common subsequence).
- D. S. Hirschberg, “A linear space algorithm for computing maximal common subsequences”, *Communications of the ACM*, 18(6), 1975.
- E. W. Myers, “An O(ND) difference algorithm and its variations”, *Algorithmica*, 1(1), 1986.
- M. T. Goodrich, R. Tamassia, *Algorithm Design*, John Wiley, 2002 (text processing, Huffman coding, LCS).
- M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 2nd ed., Pearson, 2004.